EXPERIMENT 6B : Time Series Data Analysis and Forecast

AIM: To analyze historical monthly airline passenger data, determine stationarity using the Augmented Dickey-Fuller (ADF) test, apply necessary transformations or differencing, build an ARIMA forecasting model, predict passenger demand for a test dataset, and evaluate forecast accuracy using MSE, MAE, and RMSE metrics.

ALGORITHM:

1. Import required Python libraries including pandas, numpy, matplotlib.pyplot, statsmodels.tsa.stattools (for adfuller), statsmodels.tsa.arima.model (for ARIMA), and sklearn.metrics.
2. Download the airline-passengers CSV dataset and read it into a Pandas DataFrame using parse_dates and index_col to set a proper DatetimeIndex.
3. Explore the dataset to inspect structure, missing values, and summary statistics.
4. Plot the time series to visually analyze trends, seasonality patterns, and potential outliers.
5. Perform a stationarity test by running the Augmented Dickey-Fuller (ADF) test on the series.
6. If the series is non-stationary, convert it to stationary by differencing the series.
7. Re-run the ADF test on the differenced series and visualize the resulting DataFrame.
8. If the series remains non-stationary, apply second-order differencing ($d = 2$) or log transformations before differencing.
9. Partition the original time series dataset into training and testing subsets.
10. Fit an ARIMA model on the training set using the appropriate differencing order (d) obtained from step 8.
11. Generate forecasts on the test dataset period using the trained ARIMA model.
12. Compute evaluation metrics: Mean Squared Error (MSE), Mean Absolute Error (MAE), and Root Mean Squared Error (RMSE) on the predictions.
13. Plot the training set, test set, and forecasted predictions together on a single graph for comparison.

1. Download the Airline Passengers Dataset

```
#Exp_No_6b
#Sanjay_S_241801246
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv"
df = pd.read_csv(url)
print(df.head())
```

```
     Month  Passengers
0  1949-01         112
1  1949-02         118
2  1949-03         132
3  1949-04         129
4  1949-05         121
```

2. Read the CSV with parse_dates and index_col

```
#Exp_No_6b
#Sanjay_S_241801246
df = pd.read_csv(
    url,
    parse_dates=["Month"],
    index_col="Month"
)
print(df.head())
print(df.index)
```

```
            Passengers
Month
1949-01-01         112
1949-02-01         118
1949-03-01         132
1949-04-01         129
1949-05-01         121
DatetimeIndex(['1949-01-01', '1949-02-01', '1949-03-01', '1949-04-01',
               '1949-05-01', '1949-06-01', '1949-07-01', '1949-08-01',
               '1949-09-01', '1949-10-01',
               ...
               '1960-03-01', '1960-04-01', '1960-05-01', '1960-06-01',
               '1960-07-01', '1960-08-01', '1960-09-01', '1960-10-01',
               '1960-11-01', '1960-12-01'],
              dtype='datetime64[ns]', name='Month', length=144, freq=None)
```

3. Explore the Dataset

```
#Exp_No_6b
#Sanjay_S_241801246
print("Dataset Shape:", df.shape)
```

```
Dataset Shape: (144, 1)
```

```
print("\nDataset Information:")
df.info()
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 144 entries, 1949-01-01 to 1960-12-01
Data columns (total 1 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Passengers  144 non-null    int64
dtypes: int64(1)
memory usage: 2.2 KB
```

```
print("\nStatistical Summary:")
print(df.describe())
```

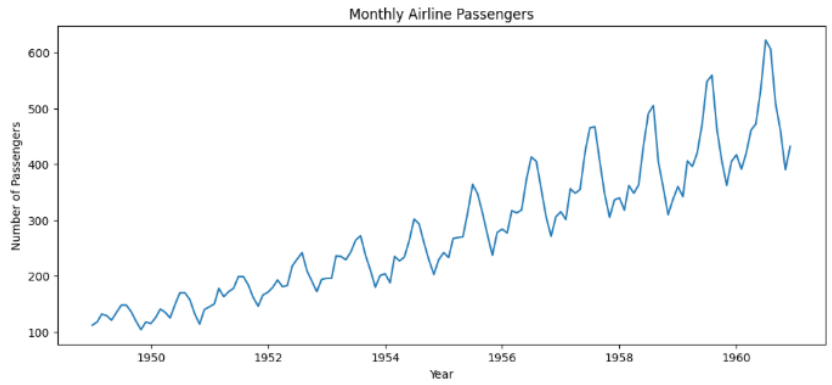
```
Statistical Summary:
       Passengers
count  144.000000
mean   280.298611
std    119.966317
min    104.000000
25%    180.000000
50%    265.500000
75%    360.500000
max    622.000000
```

```
print("\nMissing Values:")
print(df.isnull().sum())
```

```
Missing Values:
Passengers    0
dtype: int64
```

4. Plot the Series to Inspect Trend, Seasonality, and Outliers

```
#Exp_No_6b
#Sanjay_S_241801246
plt.figure(figsize=(12, 5))
plt.plot(df["Passengers"])
plt.title("Monthly Airline Passengers")
plt.xlabel("Year")
plt.ylabel("Number of Passengers")
plt.show()
```



5. Check Stationarity Using the Augmented Dickey-Fuller (ADF) Test

```
#Exp_No_6b
#Sanjay_S_241801246
from statsmodels.tsa.stattools import adfuller
result = adfuller(df["Passengers"])
print("ADF Statistic:", result[0])
print("p-value:", result[1])
if result[1] <= 0.05:
    print("The series is stationary.")
else:
    print("The series is non-stationary.")
```

```
ADF Statistic: 0.8153688792060498
p-value: 0.991880243437641
The series is non-stationary.
/tmp/ipykernel_2584/3796989905.py:2: FutureWarning: adfuller currently returns a plain tuple whose length depends on the store and autolag arguments. In release 0.16 or after July 2027, whi
  result = adfuller(df["Passengers"])
```

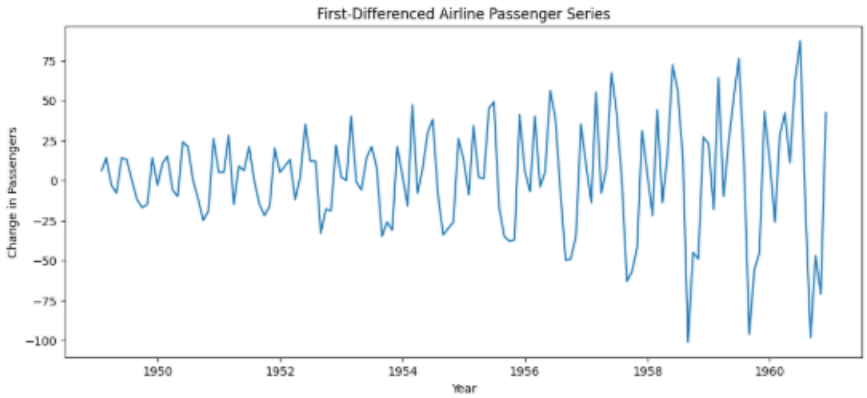
6. Convert the Non-Stationary Series into a Stationary Series Using Differencing

```
#Exp_No_6b
#Sanjay_S_241801246
df_diff1 = df["Passengers"].diff().dropna()
print(df_diff1.head())
result_diff1 = adfuller(df_diff1)
print("ADF Statistic:", result_diff1[0])
print("p-value:", result_diff1[1])
if result_diff1[1] <= 0.05:
    print("The first-differenced series is stationary.")
else:
    print("The first-differenced series is non-stationary.")
```

```
Month
1949-02-01     6.0
1949-03-01    14.0
1949-04-01    -3.0
1949-05-01    -8.0
1949-06-01    14.0
Name: Passengers, dtype: float64
ADF Statistic: -2.8292668241700047
p-value: 0.05421329028382478
The first-differenced series is non-stationary.
/tmp/ipykernel_2584/497467305.py:3: FutureWarning: adfuller currently returns a plain tuple whose length depends on the store and autolag arguments. In release 0.16 or after
  result_diff1 = adfuller(df_diff1)
```

7. Re-run ADF on the Differenced Series and Visualize It

```
#Exp_No_6b
#Sanjay_S_241801246
plt.figure(figsize=(12, 5))
plt.plot(df_diff1)
plt.title("First-Differenced Airline Passenger Series")
plt.xlabel("Year")
plt.ylabel("Change in Passengers")
plt.show()
result = adfuller(df_diff1)
print("ADF Statistic:", result[0])
print("p-value:", result[1])
```

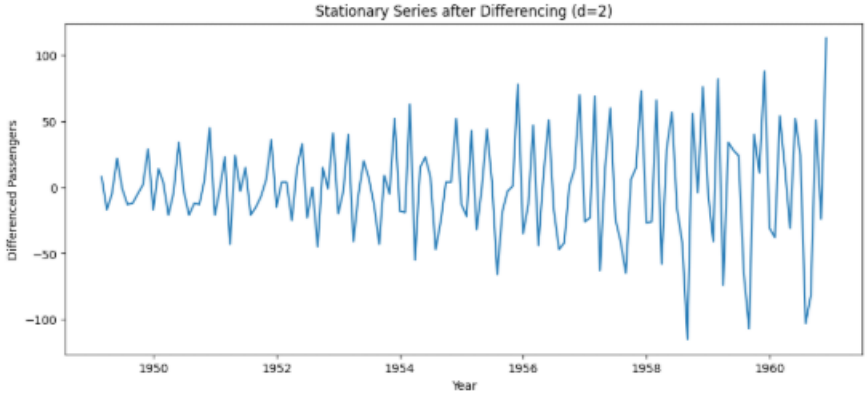



```
ADF Statistic: -2.8292668241700047
p-value: 0.05421329028382478
/tmp/ipykernel_2584/901687768.py:7: FutureWarning: adfuller currently returns a plain tuple whose length depends on the store and autolag arguments. In release 0.16 or after
  result = adfuller(df_diff1)
```

8. Apply Second Differencing if the Series Is Still Non-Stationary

```
#Exp_No_6b
#Sanjay_S_241801246
df_diff2 = df["Passengers"].diff().diff().dropna()
result_diff2 = adfuller(df_diff2)
print("Second-Difference ADF Statistic:", result_diff2[0])
print("Second-Difference p-value:", result_diff2[1])
if result_diff1[1] <= 0.05:
    d = 1
    stationary_series = df_diff1
elif result_diff2[1] <= 0.05:
    d = 2
    stationary_series = df_diff2
else:
    d = 2
    stationary_series = df_diff2
    print("Series may require further transformation.")
print("Selected differencing order (d):", d)
plt.figure(figsize=(12, 5))
plt.plot(stationary_series)
plt.title(f"Stationary Series after Differencing (d={d})")
plt.xlabel("Year")
plt.ylabel("Differenced Passengers")
plt.show()
```

```
Second-Difference ADF Statistic: -16.384231542468513
Second-Difference p-value: 2.7328918500142026e-29
Selected differencing order (d): 2
/tmp/ipykernel_2584/1860364274.py:2: FutureWarning: adfuller currently returns a plain tuple whose length depends on the store and autolag arguments.
  result_diff2 = adfuller(df_diff2)
```



9. Split the Dataset into Training and Testing Sets

```
#Exp_No_6b
#Sanjay_S_241801246
passengers = df["Passengers"]
train_size = int(len(passengers) * 0.8)
train = passengers.iloc[:train_size]
test = passengers.iloc[train_size:]
print("Training Data:", len(train))
print("Testing Data:", len(test))
```

```
Training Data: 115
Testing Data: 29
```

10. Fit the ARIMA Model on the Training Set

```
#Exp_No_6b
#Sanjay_S_241801246
from statsmodels.tsa.arima.model import ARIMA
model = ARIMA(train, order=(5, d, 0))
model_fit = model.fit()
print(model_fit.summary())
```

```
/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: No frequency information was provided, so inferred frequency MS will be used.
  self._init_dates(dates, freq)
/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: No frequency information was provided, so inferred frequency MS will be used.
  self._init_dates(dates, freq)
/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: No frequency information was provided, so inferred frequency MS will be used.
  self._init_dates(dates, freq)
                               SARIMAX Results
==============================================================================
Dep. Variable:              Passengers   No. Observations:                  115
Model:                 ARIMA(5, 2, 0)   Log Likelihood                -531.612
Date:                Sun, 11 Oct 2026   AIC                           1075.223
Time:                        06:08:08   BIC                           1091.587
Sample:                    01-01-1949   HQIC                          1081.864
                         - 07-01-1958
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1         -0.4046      0.106     -3.811      0.000      -0.613      -0.197
ar.L2         -0.5093      0.120     -4.249      0.000      -0.744      -0.274
ar.L3         -0.2456      0.081     -3.027      0.002      -0.405      -0.087
ar.L4         -0.4367      0.089     -4.885      0.000      -0.612      -0.261
ar.L5         -0.1433      0.093     -1.537      0.124      -0.326       0.039
sigma2       707.6066    104.704      6.758      0.000     502.391     912.822
===================================================================================
Ljung-Box (L1) (Q):                   0.01   Jarque-Bera (JB):                 0.33
Prob(Q):                              0.90   Prob(JB):                         0.85
Heteroskedasticity (H):               5.47   Skew:                             0.07
Prob(H) (two-sided):                  0.00   Kurtosis:                         3.22
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
```

11. Perform Forecasts on the Test Dataset

```
#Exp_No_6b
#Sanjay_S_241801246
predictions = model_fit.forecast(steps=len(test))
predictions.index = test.index
print("Forecasted Passenger Numbers:")
print(predictions)
```

```
Forecasted Passenger Numbers:
Month
1958-08-01     533.189557
1958-09-01     570.766204
1958-10-01     592.124375
1958-11-01     624.603396
1958-12-01     670.300205
1959-01-01     712.961841
1959-02-01     755.132070
1959-03-01     793.268943
1959-04-01     826.667362
1959-05-01     863.588943
1959-06-01     903.138546
1959-07-01     942.825916
1959-08-01     982.901158
1959-09-01    1021.244474
1959-10-01    1058.404551
1959-11-01    1096.393393
1959-12-01    1134.885764
1960-01-01    1173.743622
1960-02-01    1212.758558
1960-03-01    1251.207798
1960-04-01    1289.377490
1960-05-01    1327.678056
1960-06-01    1366.085996
1960-07-01    1404.677013
1960-08-01    1443.310284
1960-09-01    1481.789759
1960-10-01    1520.199332
1960-11-01    1558.609802
1960-12-01    1597.048591
Name: predicted_mean, dtype: float64
```

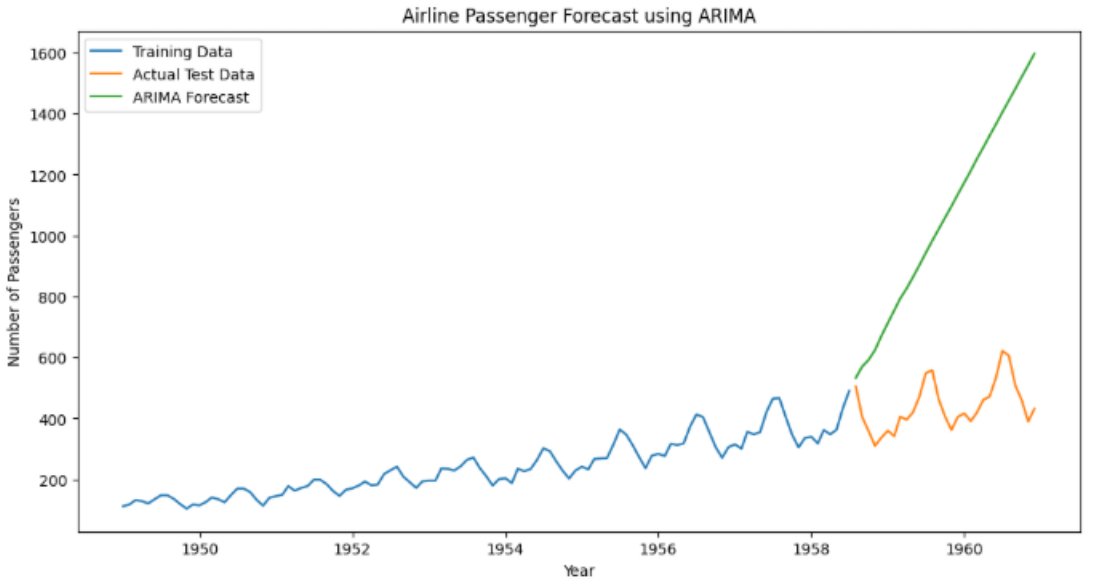
12. Compute MSE, MAE, and RMSE

```
#Exp_No_6b
#Sanjay_S_241801246
from sklearn.metrics import mean_squared_error, mean_absolute_error
mse = mean_squared_error(test, predictions)
mae = mean_absolute_error(test, predictions)
rmse = np.sqrt(mse)
print("Mean Squared Error (MSE):", mse)
print("Mean Absolute Error (MAE):", mae)
print("Root Mean Squared Error (RMSE):", rmse)
```

```
Mean Squared Error (MSE): 469187.4575741138
Mean Absolute Error (MAE): 618.6166551413303
Root Mean Squared Error (RMSE): 684.972596221275
```

13. Plot Training Data, Testing Data, and Predictions in a Single Graph

```
#Exp_No_6b
#Sanjay_S_241801246
plt.figure(figsize=(12, 6))
plt.plot(train.index, train, label="Training Data")
plt.plot(test.index, test, label="Actual Test Data")
plt.plot(predictions.index, predictions, label="ARIMA Forecast")
plt.title("Airline Passenger Forecast using ARIMA")
plt.xlabel("Year")
plt.ylabel("Number of Passengers")
plt.legend()
plt.show()
```



RESULT: Time series data analysis and forecasting on the Airline Passengers dataset were successfully implemented. The stationarity of the series was evaluated, appropriate differencing was applied, and an ARIMA model was trained, forecasted, and evaluated using MSE, MAE, and RMSE metrics alongside a visual performance plot.